FULL-STACK SYSTEM REPORT

Picturehouse Jaipur

Movie ticket booking, ride sharing, and theatre owner portal

A code-based review of the current application, its Java object model, database design, constraints, normalization, and a proposed MySQL deployment path.

CURRENT FRONTEND	CURRENT BACKEND	CURRENT DATABASE
React + Vite; Clerk email OTP; responsive light/dark UI	Java 21 + Spring Boot 3.5.6; REST API; Spring Data JPA	H2 file database for local development; MySQL is a proposed migration

Scope and accuracy

This report describes the repository as inspected on 21 October 2026. It separates implemented code from planned production work. The current project uses a file-based H2 database by default, not MySQL. Its initial catalogue is seeded development data. Razorpay integration is configured for test credentials, and ticket records/QR payloads are produced by the backend code path; actual email delivery still depends on SMTP configuration and live verification. The code does not implement WhatsApp ticket delivery or a production admission scanner.

Prepared from the checked-in React components, Java entity classes, Spring configuration, Maven manifest, package lock, and backend README.

1. Executive summary

Picturehouse Jaipur is a Jaipur-focused web application for browsing movies and theatres, selecting seats for showtimes, paying through Razorpay, retrieving a QR ticket, finding or posting shared rides, and managing theatre listings as an approved owner. It uses React for the browser interface and a Java Spring Boot API for data, authentication enforcement, payment verification, seat updates, ticket records, and owner/ride operations.

The backend is the authority for protected actions. Clerk handles email verification and session creation in the browser; Spring Security validates the bearer token before protected API calls. Razorpay creates a payment order and returns payment details; the backend validates the Razorpay HMAC signature before treating payment as captured. Movie catalogue, screens, shows, seat inventory, ride posts, ride requests, chat messages, owner memberships, orders, and ticket records are represented in JPA entities.

The product is a working development-stage application, not a fully launched ticketing service. Development seed listings are examples, seat capacity/layout and payment fulfillment need production hardening, and email/QR delivery depends on SMTP settings and successful end-to-end verification. The present report calls these out rather than implying launch readiness.

2. What the application does

- Customer experience: email OTP sign-up/sign-in with Clerk; browse Jaipur movie and theatre records; inspect showtimes and seat availability; select seats; open Razorpay Checkout; view bookings and QR credentials; search ride posts by date; post a ride, request to join, and chat around a request.

- Theatre owner experience: request access to a theatre, wait for backend approval, then use owner APIs/UI to manage theatre/movie/showtime records and view sales summaries for linked theatres.

- Security boundaries: private API routes receive a Clerk session token and are checked by the Java backend. Owner access is associated with a Clerk user ID and a theatre membership record. Admin approval is based on server-side configured Clerk IDs.

- Data and integrations: H2 file database is the local default; Razorpay is the payment provider; SMTP sends ticket email when configured; ZXing generates QR images; a Spring WebSocket broker is configured for realtime messaging infrastructure.

Important implementation distinction

H2 holds the development database and demo catalogue. The PostgreSQL JDBC driver is present as an alternate dependency, but MySQL is not currently present. `ddl-auto: update` is enabled in the current configuration; that is convenient for local development and should be replaced by controlled migrations before production.

3. Technology stack and why it fits

Layer	Current technology	Reason for use
Browser UI	React (installed 19.x family) + Vite	Component model supports the movie, ride, ticket, and owner views. Vite provides a fast local development server and production asset build.
Authentication	Clerk React SDK (`@clerk/react` 6.17.8)	Managed email OTP verification and sessions avoid building password/OTP delivery and session logic from scratch.
Backend	Java 21, Spring Boot 3.5.6	Java class model makes domain rules explicit; Spring provides REST endpoints, dependency injection, security integration, validation, mail, and WebSocket support.
Persistence	Spring Data JPA + Hibernate	Maps Java entities and relations to relational tables; repository interfaces provide standard CRUD/query operations.
Local DB	H2 in file mode	Easy local setup without a separate database server; records survive restarts. It is not intended here as the production database.
Payments	Razorpay Standard Checkout + server-side signature verification	Supports Indian payment methods through the merchant checkout; secret credentials stay on the backend.
Ticket QR	ZXing 3.5.4	Generates a QR image/payload from a backend-issued ticket credential.
Email	Spring Boot Mail / SMTP	Allows the server to send ticket confirmations and QR attachments using a configured email provider.
Build tools	npm/Vite and Maven	npm installs/builds the browser app; Maven manages Java dependencies, compilation, and Spring Boot execution.

4. Application architecture

The browser renders the product and manages the user interaction. It calls the Java API for catalog data, seat inventory, ride data, owner operations, ticket retrieval, and payment endpoints. Authentication and payment use separate trust chains: Clerk signs the user session token, while Razorpay signs the payment response. The backend verifies both before using protected identity/payment claims.

CUSTOMER / OWNER BROWSER
React + Vite UI | Clerk SDK | Razorpay Checkout
HTTPS REST + Bearer session token
JAVA API - SPRING BOOT
Controllers -> domain rules / integrations -> Spring Data JPA
Clerk JWT validation | Razorpay HMAC verification | ticket QR/email
DATABASE
H2 file now | MySQL is a proposed production target
EXTERNAL PROVIDERS: Clerk | Razorpay | SMTP provider

Core request flows

- Catalog: React -> public catalogue endpoint -> JPA repositories -> movies, theatres, screens, showtimes and seats.

- Booking/payment: authenticated customer selects seat(s) -> server creates a Razorpay order and stores pending order context -> checkout returns payment identifiers/signature -> server verifies signature and finalizes booking/seat state -> ticket record and QR are exposed to the customer.

- Owner: authenticated user asks for theatre membership -> admin approves membership -> owner endpoints check that membership before management/sales queries.

- Ride: user posts or browses a dated ride -> another user requests to join -> driver accepts/declines -> chat messages are attached to the request.

The current source includes a WebSocket broker and endpoint setup. That does not by itself establish that every seat change or ride message is pushed over a production-ready authenticated realtime channel; this should be validated separately before relying on live seat locking.

5. Object-oriented programming in the Java backend

The frontend is written mainly with React function components and hooks. The clearest demonstration of classical object-oriented design is the Java backend: entity objects hold persistent state and behavior, API controllers coordinate requests, repositories abstract persistence, and provider integrations isolate external systems.

Encapsulation

Entity fields are private. Callers use constructors, getters, and named methods instead of writing fields directly. Examples include `Showtime.reschedule(...)`, `ShowtimeSeat.markBooked()`, `RidePost.reserveSeat()`, `RideJoinRequest.accept()/decline()`, `Booking.confirm()/cancel()`, and `RazorpayOrder.markCaptured(...)`. These methods keep changes close to the state they affect. Some classes still have broad data getters and some validation lives in controllers/services, so encapsulation is present but can be strengthened by making invalid state transitions harder to express.

Abstraction

Spring Data repository interfaces express operations without exposing SQL implementation details. `RazorpayClient`, `PaymentGateway`, and `TicketDeliveryGateway` form integration boundaries around external services. The controller/service code can request an operation through these abstractions while provider details stay in adapters/configuration. Clerk itself is an external identity boundary; the backend validates signed tokens instead of reimplementing OTP delivery.

Inheritance

The application uses framework inheritance primarily through repository interfaces extending Spring Data JPA types, which supply reusable persistence behavior. Domain classes do not appear to use a custom entity base class or a deep inheritance tree. This is a deliberate, modest use of inheritance: movie, theatre, and ride concepts are related through object associations rather than forced into a shared parent class.

Polymorphism

Interfaces such as `PaymentGateway` and `TicketDeliveryGateway` allow alternative implementations to be injected by Spring. That is an extension point for swapping a test payment client for a production adapter or SMTP delivery for another delivery provider. The current application should not be described as supporting multiple live payment providers merely because an interface exists; Razorpay is the implemented payment choice. Spring also supplies repository implementations behind interfaces at runtime.

Composition and associations

A `Showtime` refers to one `Movie` and one `Screen`; a `Screen` belongs to one `Theatre`. A seat inventory row belongs to one showtime. A ride join request belongs to a ride post, and chat messages belong to a join request. Theatre/movie availability is modeled through `TheatreMovie`. This object composition mirrors the relational foreign-key relationships and keeps responsibilities understandable.

6. Current database and entity inventory

The default connection is H2 file mode at `backend/data/picturehouse` using Hibernate/JPA. H2 is a lightweight relational database suitable for local learning and development. The application seeds clearly marked sample movies, theatres, showtimes, and seat inventory unless the `prod` profile is active. The current mappings below are inferred from JPA entity annotations; physical column types and FK names are finalized by Hibernate and may vary across DB engines.

Active application tables

Table / entity	Primary key	Foreign keys / relationships	Unique/index / purpose
movies / Movie	id (identity)	None	Movie metadata; no unique title constraint.
theatres / Theatre	id (identity)	None	Jaipur theatre address, area, type, active flag.
screens / Screen	id (identity)	theatre_id -> theatres.id (required)	Unique (theatre_id, name).
showtimes / Showtime	id (identity)	movie_id -> movies.id; screen_id -> screens.id (required)	Index on starts_at; @Version for optimistic updates.
showtime_seats / ShowtimeSeat	id (identity)	showtime_id -> showtimes.id (required)	Unique (showtime_id, seat_code); booked boolean.
theatre_movies / TheatreMovie	id (identity)	theatre_id -> theatres.id; movie_id -> movies.id (required)	Unique pair (theatre_id, movie_id), resolving Theatre-Movie many-to-many.
theatre_owner_memberships / TheatreOwnerMembership	id (identity)	theatre_id -> theatres.id (required); Clerk user ID is external	Unique (clerk_user_id, theatre_id); approved, requestedAt.
razorpay_orders / RazorpayOrder	id (identity)	No JPA FK; showtime_id and Clerk user ID are scalar references	Unique provider_order_id; stores pending payment/context/status.
ticket_bookings / TicketBooking	id (identity)	No JPA FK; showtime_id and Clerk user ID are scalar references	Unique booking_reference, ticket_code, provider_order_id; email_sent_at.
ride_posts / RidePost	id (identity)	Clerk user ID, theatre/movie/show fields are scalar/snapshot fields, not FKs	Index (ride_date, active); @Version for concurrent updates.
ride_join_requests / RideJoinRequest	id (identity)	ride_post_id -> ride_posts.id (required)	Unique (ride_post_id, requester_clerk_user_id).
ride_chat_messages / RideChatMessage	id (identity)	join_request_id -> ride_join_requests.id (required)	Index (join_request_id, created_at); body max length 1200.

Legacy mapped tables

The code also contains `accounts`, `bookings`, and `carpool_rides` JPA entities. They represent an earlier in-app account/booking/carpool model; the present Clerk-backed purchase flow uses `razorpay_orders` and `ticket_bookings`, while the current ride board uses `ride_posts`, `ride_join_requests`, and `ride_chat_messages`. Because these legacy classes still carry `@Entity`, Hibernate may still create/update these tables even if current API flows do not use them. Consider removing them after confirming no migration or data still depends on them.

Primary-key and foreign-key rules

Each entity uses a generated numeric `Long` identity primary key. Ticket booking additionally has immutable business identifiers (`booking_reference`, `ticket_code`) and payment provider ID protected by uniqueness constraints. JPA `@ManyToOne(optional=false)` mappings imply required relationships and Hibernate-generated FK constraints. Membership and payment entities keep Clerk IDs as strings: Clerk remains the identity system of record, so these IDs do not point to a local users table. `RazorpayOrder` and `TicketBooking` currently store `showtime_id` as scalar IDs, so the database cannot enforce that reference as a foreign key.

Constraints currently used

- Primary keys: generated identity IDs on entity tables.

- NOT NULL: explicit `@Column(nullable=false)` declarations and required (`optional=false`) JPA associations; primitive booleans/integers are inherently non-null at the object level.

- UNIQUE: theatre/screen name; showtime/seat code; theatre/movie pair; theatre/owner pair; ride/requester pair; unique payment order, booking reference and ticket code.

- Indexes: showtime start time; ride date and active state; chat request and creation time.

- Concurrency: JPA `@Version` is present on showtimes and ride posts. A version field helps detect stale concurrent updates, but seat reservation still must be committed atomically with payment/booking and handled for duplicate payment callbacks.

- Validation: `@NotBlank` on movie title/language and theatre name/area/address; `@Email` on the legacy account email; input DTO validation and controller checks should be reviewed alongside DB constraints. Bean validation is not a replacement for NOT NULL, CHECK, or UNIQUE constraints in the database.

Notable modeling issue

Seat identifiers are stored as comma-separated strings in `RazorpayOrder.seats` and `TicketBooking.seats`. That makes a single column hold a list and prevents a relational FK/unique rule per booked seat. Ticket/order context also copies movie, theatre, and show labels as snapshots. Snapshots can be useful for receipts that must retain what was purchased even if a listing changes; they should be deliberate and paired with normalized foreign keys where appropriate.

7. Normalization assessment

Normalization is judged from dependencies and row/column structure, not from the choice of H2 versus MySQL. The central catalogue relationships (`Movie`, `Theatre`, `Screen`, `Showtime`, `ShowtimeSeat`, `TheatreMovie`) are generally organized in a 3NF-like way: theatre data is held once per theatre, screens refer to theatres, showtimes refer to a movie and screen, and seat inventory rows identify seats within a showtime. The join entity `TheatreMovie` represents the many-to-many relationship with a uniqueness rule.

However, the entire current schema should not be claimed as fully 3NF. A comma-separated seat list in one field violates First Normal Form (1NF), because it contains multiple values that should be individually addressable. Payment and ticket snapshots intentionally duplicate show/theatre/movie labels, and ride posts keep theatre/movie/show as text snapshots rather than relational references. The exact form is table-specific: the catalog core is close to 3NF, while the application as a whole is partially normalized with selected snapshot denormalization and at least one 1NF violation.

Recommended next normalization steps

- Add `ticket_booking_seats` (booking_id, showtime_seat_id or showtime_id + seat_code, price_paise). Store one row per seat. Add unique (showtime_id, seat_code) across active confirmed reservations, or rely on a unique seat reservation row and transactional state to ensure one booking per seat.

- Give `razorpay_orders` a real FK to a showtime/booking draft and store selected seat rows in a child order-seat table. Then payment verification transitions the draft to a confirmed booking and marks its exact seats.

- Link `ride_posts` to a showtime with a FK when the ride is tied to an app-listed screening. Retain a display snapshot only when the business needs historical display stability; name it as a snapshot and document why it is copied.

- Separate reusable seat map/seat category data (screen seat positions and category) from showtime seat inventory. Store dynamic show pricing in a price table with effective dates or category/show overrides; keep the paid amount as a booking-line snapshot.

- Create payment attempts/refunds and email delivery/outbox records as separate rows. This supports retries, provider events, audit history, and more than one payment/refund event per booking.

If the team later enforces 3NF throughout operational tables, every non-key attribute should depend on the key, the whole key, and nothing but the key. Snapshot copies, denormalized counters, and reporting aggregates may still be retained intentionally for audit or performance, with their source of truth and update rules made explicit. BCNF could be evaluated if non-key determinants create overlapping candidate keys; do not target a higher normal form mechanically when a justified snapshot or performance read model is more practical.

8. Review of MySQL as the next database

MySQL with InnoDB is a reasonable next step for a small commercial movie-booking application: it is widely available on managed hosting, supports transactions and relational constraints, has mature Java/JDBC support, and fits Spring Data JPA. It can store this system's catalog, seat inventory, payment records and ride data. It is a database choice, not a substitute for booking transaction design, backups, migrations, monitoring, or a verified payment flow.

Current status

The project currently defaults to H2 in `application.yml`; Maven includes H2 and PostgreSQL JDBC, but not the MySQL Connector/J dependency. The README documents PostgreSQL as an alternate, not MySQL. Switching to MySQL therefore requires adding the MySQL driver and verifying the target MySQL version, connection URL, schema migration, and production credentials. Do not paste credentials into source files or commit them.

Recommended MySQL setup and migration

- Create a managed MySQL 8.x-compatible database for the deployment region. Use InnoDB tables, `utf8mb4` character encoding/collation, a least-privilege application user, strong generated password, network allowlist, TLS, and automated backups.

- Add `com.mysql:mysql-connector-j` as a runtime dependency. Configure `DATABASE_URL` as a JDBC URL such as `jdbc:mysql://HOST:3306/picturehouse?serverTimezone=UTC`; configure user/password and `com.mysql.cj.jdbc.Driver` with deployment environment variables.

- Use a schema migration tool such as Flyway for versioned SQL. Replace production `ddl-auto: update` with `validate` or `none` after migrations are in place. Use `update` only as a local development convenience, not as a production schema-change process.

- Start with an empty staging MySQL database; run migrations and seed only verified theatre data. Test indexes, unique/FK behavior, date/time conversion, transaction boundaries and payment callback idempotency. Then perform a deliberate H2-to-MySQL export/import if existing local records need preserving.

- Use UTC for stored timestamps and convert to `Asia/Kolkata` for Jaipur display. H2's PostgreSQL compatibility mode does not prove MySQL compatibility; run integration tests against the actual MySQL engine.

Potential MySQL considerations

MySQL collations are often case-insensitive by default; choose a collation deliberately for emails, codes and titles. Verify reserved words, varchar lengths, decimal/paise representation, generated IDs, timezone conversion, and indexes under real query plans. Store money as integer paise (`BIGINT` for flexible totals) or exact `DECIMAL`, never binary floating point. Keep booking-seat allocation in a database transaction and use row-level locking or optimistic concurrency plus a unique constraint so two users cannot buy the same seat.

9. Security, payment, and ticket lifecycle

Authentication and authorization

- Clerk manages email OTP generation, delivery, verification and browser sessions. The React app uses Clerk's publishable key; it must never contain the Clerk secret key.

- For protected API requests, the frontend sends a session bearer token. Spring Security verifies the JWT signature using Clerk's JWKS and checks issuer/audience/claims as configured. The backend should derive the caller's identity from the validated token, not trust a user ID supplied in an ordinary request body.

- Theatre owner access is scoped to `theatre_owner_memberships`. Approval is a server-side decision; an owner should only manage and view sales for theatres linked to their Clerk identity and approved membership.

Payment and ticket flow

The browser uses Razorpay Checkout with a public key ID. The backend stores the private Razorpay key secret in environment configuration, creates the order, and verifies the returned signature with HMAC-SHA256. A valid signature confirms authenticity of the payment response; the backend must also verify order amount/currency/status, user ownership, selected seat availability, and replay/idempotency before finalizing. Only then should it mark seats sold and create a ticket with a unique QR credential.

The QR should contain a random, unguessable ticket token or signed ticket reference, not personal data or payment secrets. A production gate scanner needs a protected verification endpoint and atomic one-time redemption state. Email is asynchronous and can fail after ticket creation, so delivery should be retriable and recorded separately; payment success must not be reversed just because SMTP is temporarily unavailable.

Current operational limitations

- Catalogue seed data is development/example content. Replace it with approved Jaipur theatre schedules, genuine seat maps, taxes, and commercial pricing.

- Razorpay test payments are not real collections. Test keys should remain test-only; live payment requires merchant onboarding and production keys in hosting secrets.

- SMTP ticket delivery requires valid SMTP host, port, username, password and sender settings. The code has an email-delivery path, but actual delivery cannot be asserted without configuring and testing it.

- WhatsApp ticket delivery is not implemented. A compliant provider and approved message template would be a separate integration.

- An actual entry scanner/one-time QR redemption flow and robust payment webhook/reconciliation/refund handling remain production tasks.

10. Quality status and production roadmap

This report is based on source review, not a production certification. Earlier project work ran a frontend Vite production build and Maven build/tests; the Java project has no meaningful automated test suite for all payment/seat/email flows yet. A successful build demonstrates compilation, not real email arrival, payment settlement, race-free seat allocation, or live theatre accuracy.

Recommended order of work

- Choose MySQL hosting and migration strategy, add Flyway and the MySQL driver, create a staging schema, and migrate data only if required.

- Normalize seat rows into booking/order line items, implement one atomic seat reservation transaction and idempotent payment callback/webhook processing.

- Add automated Java tests for HMAC verification, duplicate callbacks, unavailable seats, concurrent buyers, owner boundary checks, and ride capacity; add MySQL integration tests using a disposable database.

- Configure a real SMTP service and verify a test message reaches the intended inbox with the QR attachment. Add retry/status tracking.

- Replace seed theatre/show/seat data with authorized accurate content; validate prices and cancellation/refund rules.

- Implement protected QR scan and one-time admission redemption; add audit logs, monitoring, backups and incident procedures.

- Run end-to-end staging checkout using Razorpay test mode first; only then complete merchant live-mode review and a carefully controlled live test.

11. Summary

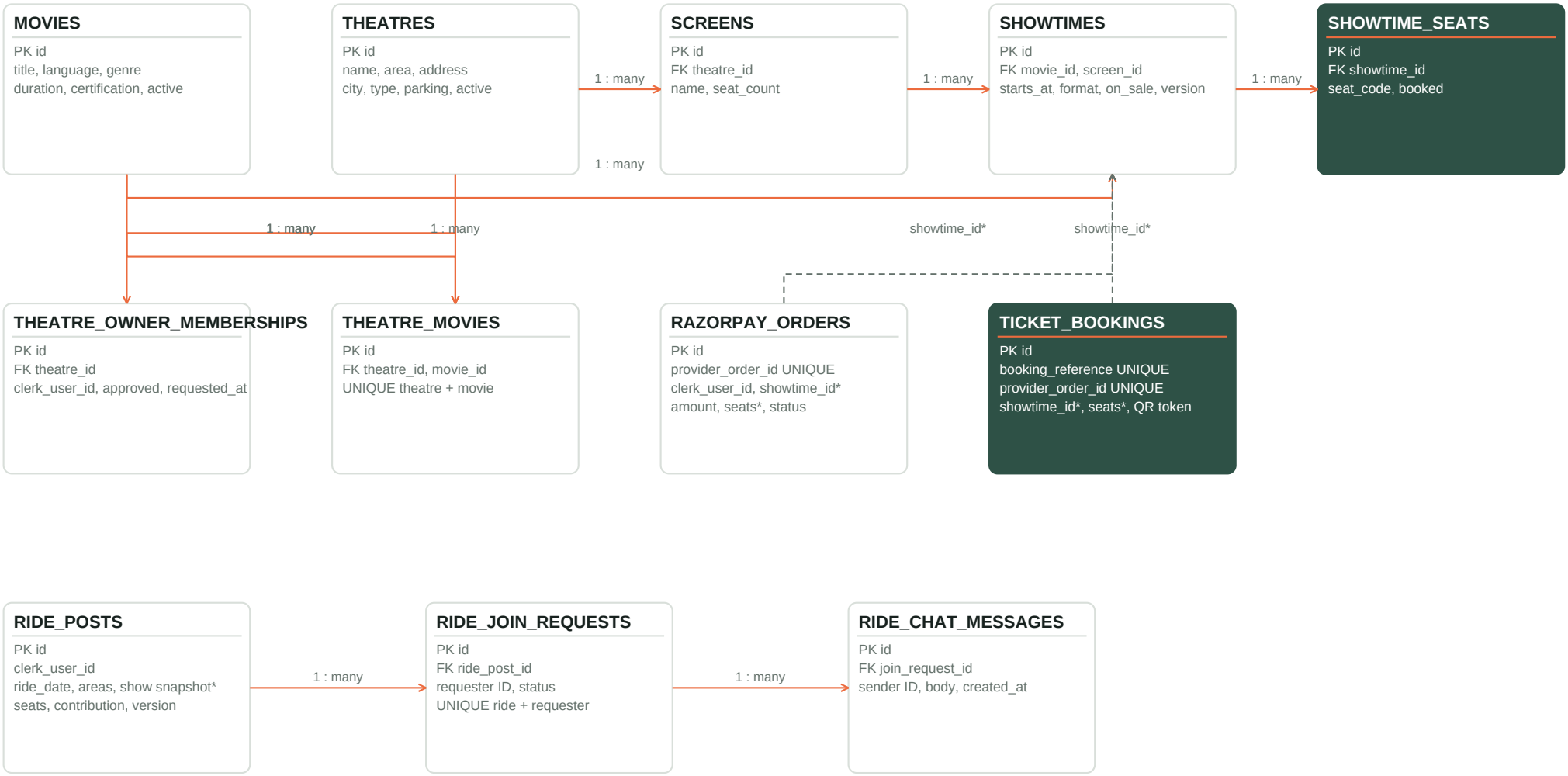
The project demonstrates a full-stack architecture with a React user experience and a Java Spring Boot backend. The Java side models key business concepts as objects and uses encapsulation, abstraction, associations, repository interfaces and Spring-managed integrations. The catalog is relational and close to 3NF, but the whole application is not fully normalized because ticket/ride snapshots are intentionally copied and selected seats are stored as comma-separated text. MySQL is a suitable next database choice, but it is a planned migration rather than the database currently in use. The ER diagram that follows describes active entity relationships and calls out references that are currently scalar/non-FK.

Repository areas reviewed

`src/App.jsx`, `src/ClerkAuth.jsx`, `src/RideBoard.jsx`, `src/OwnerPortal.jsx`, `backend/pom.xml`, `backend/src/main/resources/application.yml`, `backend/src/main/java/in/picturehouse/jaipur/domain/*`, API controllers, integration interfaces and `backend/README.md`.

ER Diagram - Current Application Model

Solid connectors are JPA/database associations. Dashed connectors are scalar references without an enforced FK. Generated ID columns are primary keys.



* Scalar/snapshot reference only in current mapping; add a real FK/child table as part of normalization. Clerk user IDs belong to Clerk, not a local users table.